Metodología de diseño

Objetivos

- Generar el entorno de desarrollo
- Reconocer la estructura básica de un programa en Java
- Comprender la diferencia entre sintaxis y semántica
- Entender el uso de plantillas sintácticas
- Ver la importancia de utilizar identificadores significativos
- Reconocer los tipos primitivos
- Conocer por qué hay diferentes tipos de datos numéricos con distintos rangos de valores
- Apreciar la diferencia entre una clase en sentido abstracto y como construcción de Java
- Ver la diferencia entre los objetos, en general, y su uso en Java
- Reconocer la diferencia entre los tipos carácter y cadena y cómo se relacionan
- Reconocer la diferencia entre constante y variable
- Saber qué es una asignación
- Comprender qué sucede cuando se invoca un método
- Conocer qué es una biblioteca
- Distinguir entre función y procedimiento
- Escribir sentencias sencillas de entrada y salida en Java

Contenido

El lenguaje Java 3
Qué necesitamos 4
Mi primer programa en Java 5
Entorno integrado de desarrollo 6
Elementos básicos de un programa 7
Sintaxis y semántica 8
Plantillas sintácticas 8
Nombres de elementos del programa: identificadores 10
Mi primer programa 11
Elementos básicos del lenguaje 14
Tipos de datos primitivos 14
Objetos y clases 17
Comentarios 20
Declaraciones 21
Expresiones y asignación 27
Asignación 27
Entrada y salida 30
Llamadas a métodos 31
Bibliotecas 33
Entrada 34
Entrada y salida interactivas 37

El lenguaje Java

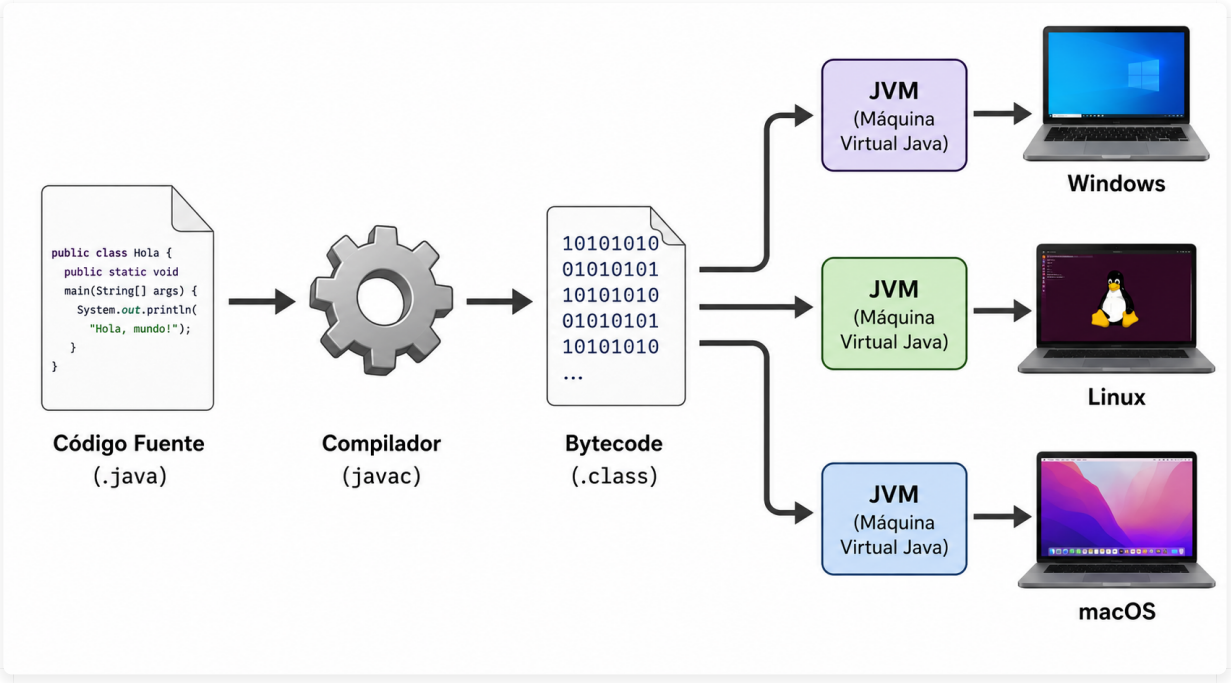
A la hora de elegir un lenguaje de programación para aprender esta disciplina encontramos muchas y variadas opciones. Hoy en día se podría considerar que, como los lenguajes más populares son *Python* o *JavaScript*, estos deberían ser nuestros principales candidatos. De hecho, *Python* es reconocido como un lenguaje de programación ideal para principiantes por tener una sintaxis clara y legible, así como un amplio conjunto de bibliotecas. *JavaScript*, por su parte, es el principal lenguaje de programación en entorno web; podría decirse que, si aprendes *HTML*, *CSS* y *JavaScript*, tienes las herramientas necesarias para hacer desarrollo web. Lenguajes como *C*, *C++* y *C#* son también muy populares e importantes, disputándose entre ellos el puesto de preferido o el de más utilizado por los programadores. En general, la elección de un lenguaje de programación u otro, no obstante, depende de los intereses, objetivos y tipo de desarrollo específico que se deba realizar.

Los siguientes enlaces lo son a diferentes listas elaboradas a partir de su popularidad, número de búsquedas en Google, utilización en forjas, etc.:

- https://www.tiobe.com/tiobe-index/
- https://pypl.github.io/PYPL.html
- https://survey.stackoverflow.co/2022/#most-popular-technologies-language
- https://spectrum.ieee.org/top-programming-languages-2022

Como se puede ver, Java se encuentra en el *Top 10* de la mayoría de clasificaciones. Entonces, ¿qué tiene Java de especial? ¿Por qué vamos a utilizarlo nosotros? Veamos algunas razones:

1. **Tiene una sintaxis clara y legible.** Java es claro y fácil de entender, lo que lo hace accesible a los programadores noveles. Facilita el proceso de aprendizaje y comprensión de los conceptos de la programación mediante una estructura de código bien organizada y un claro enfoque a la legibilidad.
2. **Orientado a objetos.** Java es un lenguaje de programación orientado a objetos (OOP) en el que los conceptos de objetos, clases, herencia, encapsulación y polimorfismo son fundamentales. La programación orientada a objetos es un paradigma ampliamente utilizado en el desarrollo de software y aprenderlo usando Java permite sentar las bases para usar otros lenguajes y conceptos relacionados.
3. **Portabilidad.** Los programas escritos en Java se pueden ejecutar en diferentes plataformas (Windows, macOS, Linux, etc.) siempre que se tenga instalado un entorno de ejecución de Java (JRE). Esta característica, denominada **WORA** (Write Once, Run Anywhere), hace que Java sea una excelente opción para desarrollar aplicaciones que funcionen en múltiples máquinas o con distintos sistemas operativos.



1. **Muchos recursos de aprendizaje.** Existe una gran comunidad de desarrolladores y una abundancia de recursos de aprendizaje: tutoriales, libros, foros, cursos en línea, etc. Esto facilita el acceso a multitud de materiales en caso de tener dudas o problemas.
2. **Versatilidad y uso extendido.** Java se utiliza ampliamente en gran variedad de dominios que incluyen el desarrollo de aplicaciones de escritorio, desarrollo web, aplicaciones móviles (Android), desarrollo de servidores y muchas más. Java proporciona una gran oportunidad para explorar diferentes áreas de la programación.

Pero, como muestra la lista del *IEEE*, el segundo lenguaje más solicitado por las empresas, justo después de *SQL*, es Java. Y Java es también el lenguaje más utilizado en las empresas de nuestro entorno. Ello, unido a las razones que hemos citado antes, convierten a este lenguaje en el mejor candidato posible para nuestros intereses.

Qué necesitamos

Para hacer un programa, en cualquier lenguaje de programación, necesitamos un editor de textos; en Windows, por ejemplo, podemos utilizar el Bloc de notas u otro como Notepad++ o Sublime Text, ambos bastante populares; lo mismo es aplicable si trabajamos con Linux o MacOS.

Por supuesto, deberíamos tener conocimientos sobre programación y el lenguaje que vamos a utilizar. En nuestro caso, se trata de ir aprendiéndolo a lo largo del curso.

Por último, necesitaremos un compilador o un intérprete[1] del lenguaje que estemos utilizando para programar. En el caso de Java, como ya indicamos, necesitamos un compilador *Java* que genera *Bytecode* y la *JVM* para ejecutarlo. Ello lo conseguiremos

1. De los que hablamos en el primer capítulo.

instalando en nuestro ordenador el **JDK** (Java Development Kit). El *JDK* es un conjunto de herramientas que incluye el compilador de Java (*javac*), la JVM (*java*) y otras utilidades necesarias para desarrollar aplicaciones en Java.

Nos dirigiremos al sitio web de Oracle (https://www.oracle.com/java[2]) y accederemos a la página de descargas[3]. Elegiremos la versión apropiada para nuestro sistema operativo y la descargaremos. Una vez completada la descarga, ejecutaremos el instalador y seguiremos las instrucciones proporcionadas por el proceso de instalación: nos pedirá seleccionar una ubicación; podemos aceptar la predeterminada o elegir una diferente.

Tras la instalación, es posible que tengamos que configurar las variables de entorno para que tu sistema reconozca la ubicación del JDK, aunque no suele ser necesario. Para comprobarlo, abre un terminal[4] y ejecuta el siguiente comando:

```
java -version
```

Deberías ver la versión del JDK instalado. En caso de no ser así, sigue los siguientes pasos:

- Windows
  1. Abre las **Propiedades del sistema**
  2. Haz clic en **Variables de entorno**
  3. Selecciona la variable **Path** y haz clic en **Editar**
  4. Agrega la ruta al directorio binario del JDK[5] al final de la lista y haz clic en **Aceptar**
- Linux y MacOS
  1. Edita el fichero **~/.bash_profile** o el **.profile** del shell que utilices.
  2. Agrega al archivo la línea:

     ```
     export PATH=/ruta/al/JDK/bin:$PATH
     ```

  3. Guarda el fichero, ciérralo y ejecuta el comando:

     ```
     source ~/.bash_profile
     ```

Mi primer programa en Java

Abre el editor de textos que prefieras y escribe lo siguiente:

```
public class MiPrimerPrograma {
    public static void main ( String[] args ) {
        System.out.println("Mi primer programa Java");
    }
```

2. También podemos acceder a la página en español: https://www.oracle.com/es/java.

3. Lo identificaremos con algún enlace o botón en el que pondrá **Descargar Java** o similar.

4. En Windows, puede ser el ***cmd***. Pulsa a la vez las teclas Windows y R: se abrirá un cuadro de diálogo con el título **Ejecutar**; escribe **cmd** en el rectángulo para introducir texto y pulsa sobre el botón **Aceptar**.

5. Como **C:\Program Files\Java\jdk-20\bin**.

```
}
```

Guarda ahora el fichero (en una carpeta o directorio que crees para ello) con el nombre **MiPrimerPrograma.java**. Es muy importante que no cambies nada y que la extensión del archivo sea **.java**, en lugar de, por ejemplo, **.txt**. Abre un terminal y accede a la carpeta o directorio en el que has guardado el fichero y ejecuta el comando:

```
javac MiPrimerPrograma.java
```

Esto utilizará el compilador de Java (**javac**) para generar un archivo ***bytecode*** llamado **MiPrimerPrograma.class** a partir del archivo fuente *Java*. Si la compilación tiene éxito[6] no se mostrará ningún mensaje de error y podremos ejecutar nuestro programa, para lo que escribiremos:

```
java MiPrimerPrograma
```

Esto ejecutará el bytecode **MiPrimerPrograma.class** y deberíamos ver en la consola la salida:

```
Mi primer programa Java
```

¡Y eso es todo! Hemos compilado y ejecutado con éxito un programa Java. Este proceso es aplicable para compilar cualquier programa escrito en Java.

Entorno integrado de desarrollo

Aunque no es objetivo de este módulo, existen unos programas denominados IDE, que integran editor de texto, compilador y depurador; y, aunque están pensados para maximizar la productividad, son, probablemente, una magnífica herramienta para un programador novel. Los IDE más populares para Java son:



- **Eclipse:** es un IDE de código abierto muy utilizado en el desarrollo de Java. Ofrece una amplia gama de características, como resaltado de sintaxis, autocompletado, depuración y soporte para la gestión de proyectos.
- **IntelliJ IDEA:** IDE desarrollado por JetBrains y conocido por su enfoque en la productividad del desarrollador. Ofrece una gran cantidad de características avanzadas, refactorización automática, integración con herramientas de construcción y control de versiones y soporte para múltiples lenguajes.

6. No debería haber problemas si lo has escrito igual.

- **NetBeans:** otro popular IDE de código abierto para el desarrollo de Java. Proporciona herramientas integradas para el desarrollo de aplicaciones web, de escritorio y móviles.
- **Visual Studio Code** con extensiones para Java, que ofrecen una experiencia ligera y altamente personalizable, ha ganado muchísima popularidad.

Todos ellos son muy capaces y ampliamente utilizados en la comunidad de desarrollo de Java. La elección del mismo depende en gran medida de las preferencias personales y necesidades específicas. Además, también es posible añadir extensiones para la programación a distintos editores de texto, como los comentados anteriormente.

Compilación y ejecución

Una vez que, con el editor, la aplicación ha quedado almacenada en un fichero (el fuente *.java*), hemos ejecutado el compilador. El compilador traduce la aplicación y almacena la versión bytecode en un nuevo fichero: el *.class*.

A veces, el compilador puede mostrar una serie de mensajes que indican errores en la aplicación. Algunos sistemas (*IDE*) permiten hacer clic en un mensaje de error para posicionar automáticamente el cursor en la ventana del editor en el punto donde se produjo el error detectado.

Si el compilador encuentra errores en el fuente (errores de sintaxis), debes determinar la causa, volver al editor y corregirlos; y ejecutar nuevamente el compilador. Una vez que la aplicación se compila sin errores, puedes ejecutarlo.

Algunos sistemas ejecutan automáticamente la aplicación cuando se compila correctamente. En otros, para ejecutar la aplicación hay que hacerlo de forma independiente. Cualquiera que sea la manera, el resultado es el mismo: el programa es cargado en memoria y ejecutado por la JVM.

Aun cuando la aplicación se ejecute, puede haber errores en el diseño. Después de todo, la computadora hace exactamente lo que le dices que haga, incluso si eso no es lo que pretendías. Si el programa no hace lo que debería (un error lógico), hay que revisar el algoritmo y corregir el código en el editor. Finalmente, compilarlo y ejecutarlo nuevamente.

Este proceso de depuración se repite hasta que la aplicación funcione según lo planeado.

Elementos básicos de un programa

Vamos a ver los elementos que necesitamos para escribir un programa que pueda realizar acciones sencillas, incluyendo entrada y salida; empezaremos explicando nuestro primer programa y lo ampliaremos.

Sintaxis y semántica

Un lenguaje de programación es un conjunto de reglas, símbolos y palabras especiales que se utilizan para construir programas. Las reglas se aplican tanto a la **sintaxis** (gramática) como a la **semántica** (significado).

La sintaxis es un conjunto formal de reglas que define exactamente qué combinaciones de letras, números y símbolos se pueden utilizar en un lenguaje de programación. La sintaxis de un lenguaje de programación no deja lugar a la ambigüedad porque el ordenador no puede pensar: no sabe lo que queremos decir. Para evitar ambigüedades, las propias reglas sintácticas deben estar escritos en un lenguaje formal, muy simple y preciso denominado **metalenguaje**.

Sintaxis: Reglas formales que determinan la construcción de sentencias válidas en un lenguaje.
Semántica: El conjunto de reglas que dan el significado de una sentencia del lenguaje.
Metalenguaje: Lenguaje utilizado para describir las reglas sintácticas de otro lenguaje.

Aprender a leer un metalenguaje es como aprender a leer un libro de instrucciones. Es cierto que muchas personas pueden aprender a hacer cosas viendo como lo hacen otros, pero no suele ser suficiente como para dominar ese tipo de actividad. Del mismo modo, podríamos aprender Java siguiendo ejemplos, pero un programador serio debe poder leer y comprender las reglas y reconocer cómo se aplican.

Las reglas sintácticas son los planos que usamos para «construir» instrucciones en un programa. Nos permiten tomar los elementos de un lenguaje, los componentes básicos del mismo, y ensamblarlos en estructuras sintácticamente correctas. Si nuestro código viola cualquiera de las reglas de ese idioma, como al escribir mal una palabra crucial o al omitir una coma importante, decimos que el programa tiene errores de sintaxis y no puede compilar correctamente hasta que los arreglemos.

Plantillas sintácticas

Una plantilla sintáctica es un ejemplo genérico de la construcción lingüística que se está definiendo. Las convenciones gráficas indican qué partes son opcionales y cuáles se pueden repetir, cuándo una palabra o símbolo debe utilizarse tal cual está en la plantilla o si un contenido puede ser reemplazado por otra plantilla.

Nosotros seguiremos las convenciones recogidas en el capítulo 2 de la especificación del lenguaje Java, aunque adaptaremos la estructura léxica para una mejor compresión.

Veamos una posible plantilla sintáctica para un programa en Java:

```
Programa:
  {ImportDeclaration}{ClassDeclaration}
```

```
ClassDeclaration:
  {ClassModifier} class Identifier { {ClassBodyDeclaration} }
```

Los datos en cursiva indican que son no terminales: referencias a otra producción. La definición del mismo viene seguida de dos puntos. Si la encontramos rodeada de unas llaves, indica que puede aparecer varias veces (incluida ninguna). Si no está en bastardilla[7], significa que es un símbolo terminal, como en el ejemplo lo son la palabra **`class`** y las llaves, que deben aparecer en el código tal cual.

Una aplicación Java puede comenzar, opcionalmente, por una serie de declaraciones de importación. Como se indicó, un lenguaje orientado a objetos (como Java) proporciona una biblioteca muy grande con objetos disponibles para que los usemos en nuestros programas. La biblioteca de Java contiene tantos, de hecho, que deben organizarse en grupos más pequeños llamados **paquetes**. Las declaraciones de importación le dicen al compilador de Java qué paquetes de la biblioteca usa nuestro programa. Veremos cómo escribir declaraciones de importación en breve pero, primero, continuemos nuestro examen de esta plantilla sintáctica.

La siguiente línea puede comenzar opcionalmente con una serie de modificadores de clase, que van seguidos de la palabra **`class`** y un identificador. Esta línea se denomina encabezado de la clase. Una aplicación en Java es una colección de elementos que se agrupan en una **clase**. El encabezado le da a la clase un nombre (el identificador) y puede opcionalmente especificar algunas propiedades generales de la clase (los modificadores de clase). Definiremos los modificadores de clase más adelante pero veamos cómo es un identificador:

```
Identifier:
  IdentifierChars

IdentifierChars:
  JavaLetter {JavaLetterOrDigit}

JavaLetter:
  any Unicode character that is a “Java letter”

JavaLetterOrDigit:
  any Unicode character that is a “Java letter-or-digit”
```

Las líneas precedentes[8] indican que un identificador es una secuencia de caracteres de identificador que, a su vez, es una letra de Java seguida, opcionalmente, de una letra o dígito de Java, que puede aparecer varias veces. Una letra de Java es una letra y, por motivos históricos, aunque se recomienda no usarlo[9], un símbolo de subrayado[10] (_) o el símbolo del dólar ($). Una letra o dígito de Java incluye, además, a los números del cero

7. Aparecerán en color rojo para distinguirlas mejor.

8. Las plantillas sintácticas que representemos, aunque obtenidas de la especificación, pueden no coincidir de forma exacta; como hemos indicado, las modificaremos para ajustarse mejor a lo que estemos explicando.

(0) al nueve (9). Las letras pueden ser mayúsculas y minúsculas, distinguiendo entre ambas.

Paquete: Colección con nombre de clases de Java que puede ser utilizada por un programa.
Clase: Definición de un objeto o programa de Java.
Identificador: Nombre asociado con procesos y objetos y se usan para referenciar a esos procesos y objetos.

El encabezado va seguido de una llave de apertura, una serie de declaraciones de clase y una llave de cierre. Estos tres elementos forman el cuerpo de la clase. Las llaves indican donde comienza y termina el cuerpo, conteniendo las declaraciones del cuerpo de clase las sentencias que le dicen al ordenador qué hacer. El programa Java más simple que podemos escribir sería algo como esto:

```
class HacerNada
{
}
```

Como su nombre indica, esto no hace absolutamente nada. Es simplemente una cáscara vacía de un programa. Nuestro trabajo como programadores es agregar instrucciones útiles a esta cáscara. Pero antes de que podamos hablar de escribir sentencias, para crear programas, debemos comprender cómo se definen los nombres en Java y familiarizarnos con algunos de los elementos del código Java.

Nombres de elementos del programa: identificadores

Usamos un identificador en Java para nombrar algo. Por ejemplo, un identificador podría ser el nombre de una clase, un subprograma (llamado **método** en Java) o un lugar en la memoria del ordenador que contiene datos (llamados **campo** en Java). Los identificadores, como acabamos de ver, se componen de letras (A–Z, a–z), dígitos (0–9), el carácter de subrayado (_) y el símbolo del dólar ($), pero cada uno debe comenzar con una letra, subrayado o símbolo del dólar.

Estos son algunos ejemplos de identificadores válidos:

```
Suma_de_esquinas J9 caja_22A TomaDatos Bin3D4 cuenta Cuenta
```

Hay que tener en cuenta que los dos últimos identificadores (cuenta y Cuenta) se consideran nombres completamente diferentes en Java. Es decir, las formas mayúsculas

9. El símbolo del dólar debe utilizarse únicamente en código generado de forma automática o, poco comúnmente, para acceder a nombres que ya existían en sistemas antiguos. El símbolo de subrayado se puede utilizar en identificadores de dos o más letras, pero no puede usarse como identificador de una letra por ser una palabra reservada. De la especificación.

10. Puede que os resulte extraño el denominarlo símbolo (o carácter) de subrayado en lugar de guion bajo o barra baja. Pero ese es su nombre.

y minúsculas de una letra son dos caracteres distintos. Estos son algunos ejemplos de identificadores no válidos y las razones por las que no lo son:

- *40Horas*: Los identificadores no pueden comenzar por un número
- *Obtener datos*: Los espacios en blanco no están permitidos en los identificadores
- *caja–22*: El guión (–) es un símbolo aritmético (resta) en Java
- *vacío_?*: Los símbolos especiales como el del cierre de la interrogación (?) no están permitidos
- *int*: La palabra `int` está predefinida en el lenguaje Java

El último identificador, `int`, es un ejemplo de **palabra reservada**. Las palabras reservadas tienen usos específicos en Java y no pueden ser usadas como identificadores definidos por el programador.

Método: Un subprograma en Java.

Campo: Una posición en memoria, referenciada por un nombre (identificador), donde se puede almacenar un objeto.

Palabra reservada: Una palabra que tiene un significado especial en el lenguaje y no puede ser utilizada como un identificador.

Uso de identificadores

Los nombres que usamos para referirnos a las cosas en nuestro código no tienen ningún significado para el ordenador: se comporta de la misma manera ya sea que llamemos a un valor 3.14159265, pi o pastel, siempre que siempre lo llamamos lo mismo. Por supuesto, para una persona es mucho más fácil darse cuenta de cómo funciona el código si los nombres elegidos para los elementos dicen algo sobre ellos. Cuando inventemos un nombre para algo, debemos siempre intentar elegir algo que sea significativo para un lector humano.

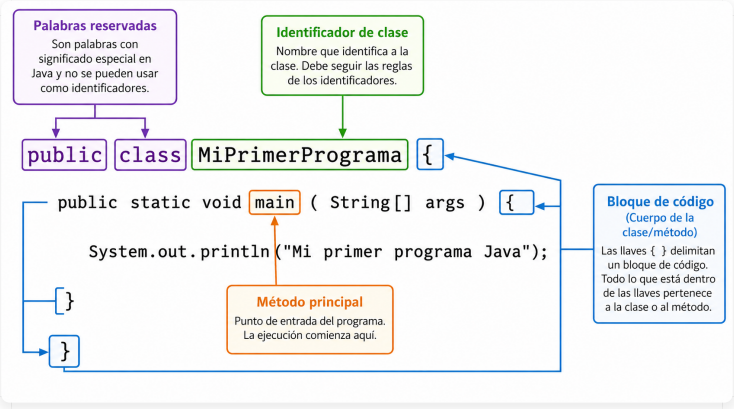
Java es un lenguaje que distingue entre mayúsculas y minúsculas, lo que significa que ve las letras mayúsculas como diferentes de las letras minúsculas. Los identificadores:

```
ITEMDATOUNO itemdatouno ITEMdatoUNO ItemDatoUno
```

son cuatro nombres distintos y no son intercambiables de ninguna manera. Podemos ver, sin embargo, que el último de estos cuatro es el más fácil de leer. Muchos programadores usan diferentes combinaciones de mayúsculas y minúsculas en los identificadores como una forma de indicar lo que representan. Más adelante veremos las convenciones habituales en Java.

Mi primer programa

Ahora estamos ya en disposición de explicar, aproximadamente, cómo funciona el código de **MiPrimerPrograma**.



El programa empieza por lo que hemos dicho que son palabras reservadas **`public`** y **`class`**. En este caso, la producción *`ImportDeclaration`* no aparece en este programa; hemos dicho que una producción entre llaves puede no aparecer. En este programa, *`ClassModifier`* está representado por **`public`**. Veremos cuáles son los modificadores de clase que podemos utilizar pero, por el momento, siempre va a ser **`public`**. Tras la palabra reservada **`class`** encontramos `MiPrimerPrograma`, que se corresponde con *`Identifier`*, el identificador de la clase y también de este programa. La plantilla sintáctica indica que, a continuación, debe haber unas llaves de apertura y cierre. Además, anidado entre ellas, puede existir un *`ClassBodyDeclaration`*.

```
ClassBodyDeclaration:
  MethodDeclaration

MethodDeclaration:
  {MethodModifier} MethodHeader MethodBody

MethodHeader:
  TypeIdentifier Identifier ( [FormalParameterList] )
```

Hemos introducido un símbolo nuevo en la plantilla de sintaxis: el corchete. Una referencia entre corchetes indica que es opcional: puede aparecer una vez o ninguna.

```
TypeIdentifier:
  Identifier

FormalParameterList:
  FormalParameter {, FormalParameter}

FormalParameter:
  TypeIdentifier Identifier
```

Esta plantilla se corresponde con esa primera línea que, por el momento, usaremos de forma literal en todos nuestros programas[11], es una declaración de método:

```
      public static void main ( String[] args ) {
```

11. En Java 21 se introduce experimentalmente una mejora que lo simplifica pero, por ahora lo seguiremos usando así. Referencias: https://openjdk.org/projects/amber/design-notes/on-ramp y https://openjdk.org/jeps/445.

Como en el caso de la declaración de clase, vemos la palabra reservada **`public`**, aunque viene aquí seguida de otro *`MethodModifier`*, **`static`**, también palabra reservada. El *`MethodHeader`* incorpora otra palabra reservada, **`void`**, y el identificador `main`: para que una clase sea un programa, uno de los métodos debe llamarse **main**. Esa línea lo que está haciendo es declarar un método `main` ya que la ejecución de una aplicación Java comienza en ese método principal[12]. Y, según la plantilla, entre paréntesis, una lista de parámetros formales.

Hasta ahora, todas las producciones tenían una única línea. La plantilla correspondiente a *`MethodBody`*, va a tener dos:

```
MethodBody:
  Block
  ;
```

Esta configuración indica que la producción puede ser una cualquiera de las líneas que aparecen: un bloque o el carácter punto y coma, en este caso.

```
Block:
  { [BlockStatements] }

BlockStatements:
  BlockStatement {BlockStatement}

BlockStatement:
  Statement

Statement:
  StatementWithoutTrailingSubstatement

StatementWithoutTrailingSubstatement:
  Block
  EmptyStatement
  ExpressionStatement

EmptyStatement:
  ;

ExpressionStatement:
  StatementExpression ;

StatementExpression:
  MethodInvocation

MethodInvocation:
  MethodName ( [ArgumentList] )

MethodName:
  ExpressionName
```

12. Si la JVM no encuentra un método **main** se lanza una excepción y finaliza.

```
ArgumentList:
  Expression {, Expression}

ExpressionName:
  Identifier
  ExpressionName . Identifier
```

Extensa secuencia de producciones sintácticas para expresar que el cuerpo del método debe ir encerrado entre llaves y va a contener sentencias que, en este caso, es una que invoca un método:

```
            System.out.println("Mi primer programa Java");
```

Este método, `println` de `System.out`, muestra por pantalla lo que hay entre los paréntesis, que es su lista de argumentos, aunque aquí solo hay uno: la frase **Mi primer programa Java**. Explicaremos más adelante que debe ir entre comillas inglesas[13] porque es un literal de cadena.

Hay que tener en cuenta que *`ExpressionName`* es una producción recursiva, es decir, que puede ser una sucesión de identificadores, sin especificar cuántos, unidos por el símbolo del punto.

Elementos básicos del lenguaje

Un programa de ordenador trabaja con datos. En Java, cada unidad de datos debe ser de un tipo de dato específico. El tipo de datos determina cómo se representan los datos en el ordenador y qué tipo de procesamiento puede realizar en ellos. Recordemos que todos los objetos son del mismo tipo, pero difieren en su clase. Java incluye tipos distintos del tipo de los objetos, cada uno de los cuales tiene su propio nombre.

Tipos de datos primitivos

Java nos los proporciona porque algunos tipos de datos se usan con mucha frecuencia. Estos se denominan tipos primitivos[14]. Estamos familiarizados con la mayoría de ellos de la vida cotidiana: números enteros, números reales y caracteres. En Java, se identifican con los tipos enteros **`int`** y **`long`**, los tipos reales **`float`** y **`double`**, los caracteres **`char`** y un tipo más: el booleano **`boolean`**.

> **Tipo primitivo:** Un tipo de datos que está disponible de forma automática para su utilización en los programas.

13. Normalmente conocidas como comillas dobles.

14. También se denominan estándar o implícitos.

Los números enteros

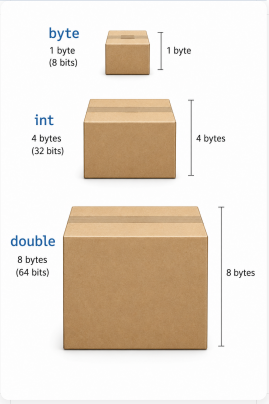
Son todos los números positivos y negativos (sin parte fraccional). Constan de un signo y dígitos[15]:

```
+57 -45 3 987 -34 0 32768
```

Teóricamente no hay límite sobre el tamaño de los números pero el hardware de la máquina y consideraciones prácticas sí imponen limitar el tamaño de un entero. La mayoría de los lenguajes definen varios tipos de enteros, que se diferencian fundamentalmente en el número de octetos (**bytes**) que se usan para su representación. Cuantos más octetos usemos para representar un número entero, mayor será el rango de números representable.

Por ejemplo, en Java, existen cuatro tipos de números enteros:

Nombre del tipo	Tamaño usado (en bits) para su representación	Total de números distintos representables
`byte`	8 (1 octeto)	$2^8 = 256$
`short`	16 (2 octetos)	$2^{16} = 65.536$
`int`	32 (4 octetos)	$2^{32} = 4.294.967.296$
`long`	64 (8 octetos)	2^{64}



Al representar números positivos y negativos, además del cero, los valores que pueden tener son la mitad del total[16]. En la actualidad, es habitual definir todos los números enteros como de tipo **`int`**, aunque el valor que vayan a tomar pueda ser representado con un **`byte`** o un **`short`**[17]. Y solo si tenemos seguridad de que el valor que pueda tomar es muy grande (más de dos mil millones), usaremos **`long`**.

Números reales

Son números decimales, es decir, tienen una parte entera y una fraccionaria, con un punto decimal entre ellos. Podemos usar también la notación científica: un valor decimal multiplicado por una potencia de diez que indica la posición real del punto decimal. En lugar de escribir 1.876×10^{12}, utilizaremos el formato 1.876e12.

Por ejemplo, en Java, se definen como de tipo real los siguientes:

---

15. Si se omite el signo se asume que el número es positivo.

16. La mitad en los números negativos y la mitad menos uno en los positivos, además del cero.

17. Los utilizaremos, por ejemplo, en caso de necesitar una gran cantidad de datos y pueda afectar a los requerimientos de memoria.

Nombre del tipo	Tamaño usado (en bits) para su representación	Total de números distintos representables
`float`	32 (4 octetos)	2^{32} = 4.294.967.296
`double`	64 (8 octetos)	2^{64} = 1.844.674.407 E+19

En el caso de los números reales, cuando no tengamos limitaciones serias de la memoria disponible para ejecutar nuestra aplicación, se recomienda usar siempre el tipo de mayor precisión, ya que así, al operar, los errores cometidos por las sucesivas aproximaciones serán menores.

El tipo booleano

Booleano es un tipo con solo dos valores: `true` y `false`. Este tipo se asocia con los datos creados dentro del subprograma y usados para representar la respuesta a preguntas. La importancia de este tipo de datos se entenderá mejor cuando estudiemos las condiciones y cómo se toman decisiones por parte de la máquina. Este tipo de datos recibe su nombre por George Boole (1815-1864), matemático inglés que creó un sistema lógico usando variables con solo dos valores.

El tipo de datos char

El tipo primitivo **`char`** describe datos que consisten en un carácter alfanumérico: una letra, un dígito o un símbolo especial. Java utiliza un juego de caracteres particular o conjunto de caracteres que puede representar. El juego de caracteres de Java, que se llama Unicode, incluye caracteres para muchos idiomas escritos. Lo normal es que nosotros usemos un subconjunto de Unicode que corresponde a la codificación ASCII. ASCII consta del alfabeto inglés[18], más números y símbolos.

Aquí hay algunos valores de ejemplo de tipo **`char`**:

```
'A' 'a' '8' '2' '+' '-' '$' '?' '*' ' '
```

Observemos que cada carácter está encerrado entre apóstrofos[19] (comillas simples). El compilador de Java necesita las comillas para poder diferenciar entre los datos de tipo caracter[20] y otros elementos de Java. Por ejemplo, los apóstrofos alrededor de los caracteres `'A'` y `'+'` los distinguen del identificador *A* y del símbolo de la suma. Notemos también que el espacio en blanco, `' '`, es un caracter válido.

¿Cómo escribimos el apóstrofo como un caracter? Si escribimos `'''` el compilador de Java se quejará de que hemos cometido un error de sintaxis: el segundo apóstrofo indica

18. Podemos utilizar la eñe y las letras con tilde, pero, en general, no se recomienda esta opción.

19. Es frecuente confundir este término con apóstrofe, que tiene un significado diferente.

20. El término caracter, sin tilde y acentuado en su última sílaba, es de uso común en la jerga informática española para diferenciar el tipo de datos de los símbolos a los que representa. Es por ello que, salvo que la corrección automática lo haya modificado, aparecerá así en el texto al hacer referencia al tipo de datos.

el final de un carácter (el vacío) y el tercero inicia un nuevo valor de caracter. Para solucionar este problema, Java proporciona lo que se denomina una secuencia de escape que nos permite escribirlo como un caracter. Es decir, Java trata la secuencia de dos caracteres `\'` como un solo caracter que representa el apóstrofo. Cuando queramos representar el apóstrofo como un caracter de Java, deberemos escribirlo así:

```
'\''
```

Es importante notar que usamos la barra inversa[21] (\) como carácter de escape y no la barra[22] (/). Java usa el caracter barra como símbolo de la división, por lo que es importante reconocer que ambas son diferentes. Pensando en ello un momento, este esquema introduce un nuevo problema: ¿cómo escribimos la barra inversa? La respuesta es que Java proporciona una segunda secuencia de escape, \\, que permite escribirla. Por lo tanto, escribimos el valor de **`char`** de la barra inversa en Java de la siguiente manera:

```
'\\'
```

No debemos confundir esta secuencia de caracteres con la secuencia //, que comienza un comentario en Java (veremos los comentarios un poco más adelante).

Java proporciona operaciones que nos permiten comparar valores de datos de tipo **`char`**. El conjunto de caracteres Unicode está ordenado en lo que se conoce como secuencia de cotejo: un orden predefinido para todos los caracteres. En Unicode, `'A'` se compara como menor que `'B'`, `'B'` como menor que `'C'`, y así sucesivamente. Además, `'1'` se compara como menor que `'2'`, `'2'` como menor que `'3'` y, así, sucesivamente.

El tipo **`char`** es uno de los tipos primitivos de Java. La clase `String`, que nos permite trabajar con colecciones de caracteres, como palabras y oraciones, es uno de los tipos de objetos en Java.

Objetos y clases

Identificamos en el capítulo anterior dos fases de programación: la fase de resolución de problemas y la fase de implementación. A menudo, utilizamos el mismo vocabulario de diferentes maneras en las dos fases.

21. Aunque este es el que define la RAE, en la literatura, podemos encontrarla denominada como barra invertida, contrabarra…

22. Análogamente, también tiene otras denominaciones.



En la fase de resolución de problemas, por ejemplo, un **objeto** es una entidad o algo que tiene sentido en el contexto del problema en cuestión. Un grupo de objetos, con propiedades y comportamientos similares, se describen como una clase de objeto o **clase**, para abreviar. La resolución de problemas orientados a objetos implica aislar los objetos que componen el problema. Los objetos interactúan unos con otros mediante el envío de mensajes.

En la fase de implementación, una **clase** es una construcción que permite al programador describir un objeto. Una clase contiene campos (valores de datos) y métodos (subprogramas) que definen el comportamiento del objeto. Podemos pensar en una clase, en sentido general, como en un patrón para un objeto, qué parece y cómo se comporta; y en una clase de Java como la estructura que nos permite simular el objeto en código. Si una clase es una descripción de un objeto, ¿cómo obtenemos un objeto? Usamos un operador, llamado `new`, que toma el nombre de la clase y devuelve un objeto de esa clase. El objeto que se devuelve es una instancia de la clase. La acción de crear un objeto a partir de una clase se denomina **instanciación**.

Las siguientes definiciones proporcionan un nuevo significado a los términos, que se añaden a los que ya definimos con anterioridad:

Clase (en sentido general): Descripción del funcionamiento de un grupo de objetos que tienen las mismas propiedades y comportamiento.
Clase (en Java): Plantilla para un objeto.
Objeto (en sentido general): Cosa o entidad que es relevante en el contexto de un problema.
Objeto (Java): Instancia de una clase.
Instanciación: Creación de un objeto, instancia de una clase.
Método: Subprograma que define algún aspecto del comportamiento de una clase.

La posible plantilla para una aplicación Java que mostramos con anterioridad podemos usarla como plantilla de declaración de clase porque una aplicación Java es solo una clase que tiene un método llamado `main`. Por ahora solo nos interesan los aspectos para definir una clase que se aplican a todas las clases de Java.

La clase String

Mientras que un valor de tipo **`char`** está limitado a un solo carácter, una cadena (en el sentido general) es una secuencia de caracteres, como una palabra, un nombre o una oración, encerrada entre comillas inglesas. En Java, una cadena es un objeto, una instancia de la clase `String`. Por ejemplo:

```
"Introducción a Java" "Tema 2" "Programción" "."
```

Una cadena debe escribirse en una única línea. Por ejemplo:

```
"Esta cadena no es válida porque
está escrita en más de una línea."
```

no es válida porque está dividida en dos líneas antes del símbolo de comillas de cierre. En esta situación, el compilador de Java emitirá un mensaje de error en la primera línea. El mensaje mostrará algo como *QUOTE EXPECTED*, o similar.

Las comillas no se consideran parte de la cadena: están ahí para distinguirlas. Por ejemplo, `"amor"` (entre comillas) es la cadena de caracteres formada por las letras *a*, *m*, *o* y *r*, en ese orden; por otra parte, `amor` (sin las comillas) es un identificador. Los símbolos `"123"` representan una cadena formada por los caracteres *1*, *2* y *3*, en ese orden. Si escribimos `123`, sin las comillas, es una cantidad entera (el número ciento veintitrés) que se puede utilizar en los cálculos.

Una cadena que no contiene caracteres se denomina la cadena vacía. Escribimos la cadena vacía usando dos comillas inglesas sin nada (ni siquiera espacios) entre ellas:

```
""
```

La cadena vacía no es equivalente a una cadena de espacios; más bien, es una cadena especial que no contiene caracteres.

Para escribir el símbolo de comillas dentro de una cadena, usamos una secuencia de escape: `\"`. Ejemplo de cadena que contiene comillas y barra inversa:

```
"Ella dijo: \"No olvides que '\\' es distinto de '/'.\""
```

Cuyo valor es:

```
Ella dijo: "No olvides que '\' es distinto de '/'."
```

Fijémonos en que dentro de una cadena no tenemos que usar la secuencia de escape `\'` para representar un apóstrofo. Análogamente, podemos escribir las comillas dobles como un valor de tipo **`char`** (`'"'`), sin usar una secuencia de escape. Por el contrario, tenemos que usar `\` para escribir una barra inversa como un valor de caracter o dentro de una cadena.

Java proporciona operaciones para unir y comparar cadenas, copiar porciones de las mismas, convertir mayúsculas en minúsculas y viceversa o convertir números a cadenas y al revés. Veremos estas operaciones más adelante.

Comentarios

El compilador ignora los comentarios, pero este tipo de documentación es de enorme ayuda para cualquiera que deba leer el código. Los comentarios pueden aparecer en cualquier parte excepto en medio de un identificador, una palabra reservada o una constante literal.

Los comentarios de Java pueden escribirse de dos formas. El primero es cualquier secuencia de caracteres encerrada entre el par `/* */`. El compilador ignora cualquier cosa que esté dentro del mismo. Aquí hay un ejemplo:

```
String numeroId; /* Número de identificación de la aeronave */
```

Este tipo de comentarios se denominan de bloque y pueden ocupar más de una línea. Cuando el primer carácter del comentario es un asterisco, el comentario tiene un significado especial que indica que debe ser utilizado por un programa de generación automática de documentación llamado *javadoc*. Veremos también lo útil que resulta que los comentarios empiecen con `/**`.

La segunda forma, y más utilizada, comienza con dos barras inclinadas (//) y se extiende hasta el final de la línea:

```
String numeroIdAeronave; // Identificación de la aeronave
```

Este tipo de comentarios se denominan de línea y el compilador ignora cualquier cosa que esté escrita después de las dos barras hasta el final de la línea.

Escribir código completamente comentado es un buen estilo de programación. Debería aparecer un comentario al comienzo del programa o clase para explicar lo que hace:

```
/*
   Este programa calcula el peso y el centro de masas de un
   avión, dada la cantidad de combustible, número de pasajeros
   y peso del equipaje en cada bodega.
   Se supone que hay dos pilotos y que los pasajeros pesan, en
   media, 80 kilos cada uno.
*/
```

Otro lugar idóneo para los comentarios son las declaraciones de campo, donde los comentarios pueden explicar cómo se utiliza cada identificador. Además, deben explicar cada paso principal en un segmento de código y cualquier cosa que sea inusual o difícil de entender (por ejemplo, una fórmula larga).

Los comentarios deben ser concisos y organizarlos para que sean fáciles de ver y quede claro lo que documentan. Si los comentarios son demasiado largos o abarrotan las declaraciones, pueden llegar a hacer el código más difícil de leer, al contrario de lo que se pretende.

Los comentarios no se necesitan para explicar las sentencias individuales, deben explicar los algoritmos.

Declaraciones

¿Cómo le decimos al ordenador qué representa un identificador? Usamos una **declaración**: una declaración asocia un nombre (un identificador) con una descripción de un elemento en Java (al igual que una definición de diccionario asocia un nombre con una descripción de la cosa a la que nombra). En una declaración, nombramos tanto el identificador como lo que representa.

> **Declaración:** Sentencia que asocia un identificador con un elemento de forma que el usuario se puede referir a ese elemento por un nombre.

Cuando declaramos un identificador, el compilador elige una ubicación en la memoria para ser asociado con ello. No tenemos que saber la dirección real de la ubicación de la memoria porque el ordenador realiza un seguimiento automáticamente por nosotros.

Para ver cómo funciona este proceso, suponga que, cuando enviamos una carta, solo tenemos el nombre del destinatario y la oficina de correos busca la dirección por nosotros. Por supuesto, todos en el mundo necesitaríamos tener un nombre diferente con tal sistema; de lo contrario, la oficina de correos no podría averiguar cuál es la dirección. Lo mismo es cierto en Java. Cada identificador puede representar una sola cosa (excepto en circunstancias especiales, que ya veremos). Cada identificador del código debe ser diferente de todos los demás.

Declaración de campo

Las clases se componen de métodos y campos. Los campos son los componentes de una clase que representan los datos. Los datos de una clase pueden ser de cualquier tipo, incluidos los tipos primitivos o los objetos. Es importante comprender la importancia de poder tener objetos dentro de objetos, ya que permite ir construyendo, poco a poco, objetos de gran complejidad.

Usamos identificadores para referirnos a los campos. En Java debe declararse cada identificador antes de que sea usado. El compilador puede entonces verificar que el uso del identificador es consistente con su declaración. Si declaramos que un identificador es un campo que puede contener un valor `char` y luego intentamos almacenar un número, por ejemplo, el compilador detectará esta inconsistencia y emitirá un mensaje de error.

Un campo puede ser una constante o una variable. En otras palabras, un identificador de campo puede ser el nombre de una ubicación de memoria cuyo contenido no puede

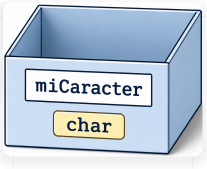
cambiar o puede ser el nombre de una ubicación de memoria cuyo contenido puede cambiar. Hay diferentes tipos de sentencias de declaración de variables y constantes en Java. Vamos primero a ver las variables, luego las constantes y, finalmente, los campos en general. Más adelante veremos cómo declaramos métodos y clases.

Variables

Los datos se almacenan en la memoria. Mientras se ejecuta una aplicación, diferentes valores pueden almacenarse en la misma posición de memoria en diferentes momentos. Estrictamente hablando, la posición de memoria se denomina **variable** y su contenido es el valor de la variable. El nombre simbólico que asociamos con la posición de memoria es el nombre de la variable o identificador de variable. En la práctica, llamamos variable al nombre de la variable y lo que se dice que cambia es su valor.

Declarar una variable implica especificar tanto su nombre como su tipo de datos o clase. Esto le dice al compilador que asocie un nombre con una posición de memoria y le informa que sus contenidos serán de un tipo específico o clase (por ejemplo, `char` o `String`). La siguiente declaración indica que `miCaracter` será una variable de tipo `char`:

```
char miCaracter;
```



Debemos tener en cuenta que la declaración no especifica qué valor se almacena en la variable, solo que puede contener un valor de tipo `char`. `miCaracter` ha sido reservado como un lugar en la memoria pero no contiene datos.

Java es un lenguaje **fuertemente tipado**, que significa que solo los valores de los datos del tipo o clase especificado en su declaración pueden almacenarse en dicha variable. Debido a la declaración anterior, la variable `miCaracter` solo puede contener un valor de tipo caracter. El compilador de Java hará una comprobación para ver que no se escriban instrucciones que intenten almacenar un valor del tipo incorrecto. Si se hace, dará un mensaje de error usualmente diciendo algo como *CANNOT ASSIGN STRING TO CHAR*.

Variable: Una posición en memoria, referenciada por un nombre de variable (identificador), donde se puede almacenar un valor del dato (este valor puede cambiarse).

Fuertemente tipado: Propiedad de un lenguaje de programación en el que las variables solo pueden contener valores del tipo especificado (para ellas).

Plantilla sintáctica de declaración de variable:

```
VariableDeclaration:
  {TypeModifier} TypeIdentifier IdentifierList ;

IdentifierList:
  Identifier {, Identifier}
```

Por el momento, el modificador de tipo, como en las declaraciones de clase, es **public** (o nada) y el identificador de tipo es el nombre de un tipo o clase (como **int** o `String`). Iremos viendo más modificadores a medida que los necesitemos, teniendo en cuenta que son opcionales: podemos escribir una declaración sin usar ninguno de ellos. Tenemos que tener en cuenta también que una sentencia de declaración siempre termina con un punto y coma.

En la plantilla sintáctica podemos ver que resulta posible declarar varias variables en una misma sentencia:

```
char letra, inicial, ch;
```

Esta declaración le dice al compilador que reserve posiciones de memoria para tres variables de tipo **char**. Sin embargo, en general, preferimos declarar cada variable en una sentencia separada:

```
char letra;
char medio;
char ch;
```

Hacerlo de esta manera permite adjuntar comentarios a la derecha de cada declaración. Por ejemplo:

```
String nombre;   // El nombre de una persona
String apellido; // El apellido de una persona
String titulo;   // El título de una persona, como Dr.
char inicial;    // Inicial del nombre de una persona
char miCaracter; // Un lugar para almacenar una letra
```

Estas declaraciones le dicen al compilador que reserve espacio de memoria para tres variables cadena y dos variables caracter. Los comentarios explican qué representa cada variable a alguien que lea el programa; iremos viendo que, dado que los identificadores pueden -y deben- ser significativos, ese tipo de comentarios será redundante y, por tanto, innecesario.

Ampliamos dos de las producciones que veíamos antes acerca del contenido de clases y bloques:

```
ClassBodyDeclaration:
  ClassMemberDeclaration

ClassMemberDeclaration:
  Declaration
  MethodDeclaration
```

```
BlockStatement:
  Declaration
  Statement

Declaration:
  VariableDeclaration
```

Como vemos, una clase contiene un conjunto de declaraciones de clase, algunas de las cuales pueden ser variables. Por ejemplo, la siguiente clase tiene dos declaraciones de variable:

```
public class Muestra // Encabezado de una clase pública llamada
                     // Muestra
{
    char miCaracter; // Variable de tipo char declarada
                     // en la clase Muestra
    String miCadena; // Variable de objeto String
                     // declarada en la clase Muestra
}                    // Final de la clase Muestra
```

Ahora que hemos visto cómo declarar variables en Java, veamos cómo declarar constantes.

Constantes

Todos los caracteres individuales (entre apóstrofos) y cadenas (entre comillas dobles) son constantes.

```
'A' '@' "Hola alumnado" "Por favor, ingrese su talla:"
```

En Java, como en matemáticas, una constante es algo cuyo valor es fijo, nunca cambia. Cualquier valor real de una constante en un programa se denomina un **valor literal** o, simplemente, literal.

En vez de escribir cada constante cada vez que aparece, se puede hacer de una forma más legible dando a cada constante un nombre y usando dicho nombre en el código. Tales constantes se denominan **constante con nombre**. Por ejemplo, podemos escribir una instrucción que imprima el nombre del ciclo usando la cadena literal `"Desarrollo de Aplicaciones Multiplataforma"`, o podemos declarar una constante con nombre llamada `TITULO_CICLO` que es igual a esa cadena y luego usar el nombre constante en la instrucción. Es decir, podemos usar cualquiera de las dos:

```
"Desarrollo de Aplicaciones Multiplataforma"
```

o

```
TITULO_CICLO
```

Usar el valor literal de una constante puede parecer más fácil que darle un nombre y luego referirse a ella con ese nombre. Pero las constantes con nombre hacen que un

programa sea más fácil de leer, porque aclaran el significado de los literales, y facilitan cambiar más adelante el programa.

Valor literal: Cualquier valor constante escrito en un programa.
Constante con nombre: Una posición en memoria, referenciada por un nombre de constante (identificador), donde se almacena un valor de dato (este valor no puede ser cambiado).

La plantilla sintáctica para una declaración de constante es similar a la plantilla para una declaración de variable:

```
ConstantDeclaration:
  {TypeModifier} final TypeIdentifier Identifier = Literal ;
```

La única diferencia es que debemos incluir el modificador **final**, una palabra reservada, seguido del identificador, el símbolo de igualdad (**=**) y el valor que se almacenará en la constante. El modificador **final** le dice al compilador de Java que este valor es el único valor que debe tener este identificador.

```
Declaration:
  ConstantDeclaration
  VariableDeclaration
```

Los siguientes son ejemplos de declaraciones de constantes:

```
final String ASTERISCOS = "********";
final char BLANCO = ' ';
final String TITULO_CICLO =
     "Desarrollo de Aplicaciones Multiplataforma";
final String MENSAJE = "Condición de error";
```

Como se muestra en el código anterior, muchos programadores de Java escriben con mayúscula el identificador completo de una constante con nombre y separan las palabras con un símbolo de subrayado. La idea es ayudar al lector a distinguir rápidamente entre nombres de variables y nombres de constantes cuando aparecen en medio del código. En uno de los casos hemos dividido la declaración en dos líneas, colocando la cadena literal en la línea que sigue a la definición del identificador. Esto es válido en Java (porque la declaración finaliza con el punto y coma), pero no podemos dividir la cadena literal en dos líneas.

Es una buena idea agregar comentarios a las declaraciones de constantes, como hacíamos con las declaraciones de variables, siempre que aporten información. Por ejemplo:

```
final String ASTERISCOS = "********"; // Para separador
final char BLANCO = ' ';              // Un espacio en blanco
```

Es extremadamente recomendable el uso de constantes con nombre en lugar de literales, siempre que las constantes tengan algún significado útil asociado.

Campos

La similitud que existe entra la declaración de constantes y variables no es una coincidencia. Java, en realidad, no distingue entre las declaraciones de constantes con nombre y las de variables porque ambas se consideran solo diferentes tipos de campos. Una constante con nombre es simplemente un campo con el modificador **`final`**, que dice que el valor nunca puede cambiar. Si ampliamos la plantilla para una declaración de variable para incluir la sintaxis necesaria para dar a la variable un valor inicial y añadir la palabra clave **`final`** a la lista de modificadores, entonces tenemos una plantilla genérica para una declaración de campo en Java:

```
FieldDeclaration:
  {TypeModifier} TypeIdentifier IdentifierList ;

IdentifierList:
  Identifier [= Literal] {, Identifier [= Literal]}

Declaration:
  ConstantDeclaration
  FieldDeclaration
```

Las siguientes declaraciones son legales:

```
final String PALABRA1 = "Ciclo de Grado Superior de";
      String palabra3 = "Desarrollo de Aplicaciones";
final String PALABRA5 = "Multiplataforma";
```

Almacenan `"Ciclo de Grado Superior de"` como el valor de la constante `PALABRA1`; la cadena `"Desarrollo de Aplicaciones"` se almacena en la variable `palabra3`; y `"Multiplataforma"` en la constante `PALABRA5`.

Capitalización de identificadores

Los programadores utilizan una serie de reglas para escribir los identificadores de manera que, al verlos, proporcionen rápidamente la idea de qué representan. Distintos programadores utilizan convenciones diferentes en el uso de mayúsculas y minúsculas. En algunos lenguajes se dan recomendaciones o existen por tradición; por ejemplo, en C, (casi) todo se escribe en minúsculas y se separa con el carácter de subrayado.

En Java, distintas comunidades han propuesto y establecido convenciones para los identificadores; nosotros seguiremos las de Sun Microsystems. Los nombres se

construyen utilizando palabras significativas, sin abreviaturas; igualmente, se evitan los acrónimos, utilizando palabras completas sin separar entre ellas, siguiendo un estilo *CamelCase*[23].

Las clases son sustantivos en *UpperCamelCase* (la primera letra en mayúsculas), como en `Ejemplo` o `MiClase`. Los métodos son verbos en *LowerCamelCase* (la primera letra en minúsculas), como en `pintar` o `pagarIntereses`. Las variables deben también ir en *LowerCamelCase* y no comenzar por el carácter de subrayado o el símbolo del dólar. Como ya hemos indicado, (estas palabras) deben ser significativas, como un mnemónico. Deben evitarse las variables de un único carácter excepto para temporales, como veremos más adelante. Las constantes con nombre deben utilizar mayúsculas, separando sus palabras componentes con símbolos de subrayado.

Expresiones y asignación

Hasta ahora, hemos visto formas de declarar campos en una clase. Como parte de la declaración, podemos dar un valor inicial al campo.

Asignación

Podemos establecer o cambiar el valor de una variable a través de una *sentencia de asignación*. Por ejemplo:

```
apellido = "Cepero";
```

asigna el valor de cadena `"Cepero"` a la variable `apellido` (es decir, almacena la secuencia de caracteres `"Cepero"` en la posición de memoria asociada con la variable denominada `apellido`).

Esta es la plantilla sintáctica para una sentencia de asignación:

```
Assignment:
  LeftHandSide AssignmentOperator Expression

LeftHandSide:
  Identifier

AssignmentOperator:
  =
```

La semántica (significado) del operador de asignación (=) es «es igual a» u «obtén»; la variable obtiene el valor de la **expresión** (*`Expression`*). Cualquier valor anterior de la variable se reemplaza por el de la expresión. Podemos observar que la sintaxis es la misma que para asignar un valor inicial a un campo en una declaración de campo.

23. *CamelCase* es un término en inglés que hace referencia a que las primeras letras de una palabra se escriben en mayúsculas, asemejándose a las jorobas de un camello.

Además, podemos ampliar la plantilla (ya vista con anterioridad) *StatementExpression*, que definía una sentencia únicamente como la invocación de un método:

```
StatementExpression:
  Assigment
  MethodInvocation
```

Solo un identificador de variable puede aparecer en el lado izquierdo de una sentencia de asignación. La asignación no es como una ecuación matemática tal como x + y = z + 4; la expresión, lo que está en el lado derecho del operador de asignación, se **evalúa** y ese valor se almacena en la variable que está en el lado izquierdo del operador de asignación. Las variables toman el valor que se les asigna hasta que se cambian mediante otra sentencia de asignación.

Sentencia de asignación: Acción que da el valor de una expresión a una variable.

Expresión: Conjunto de variables, constantes y operadores que puede ser evaluado para obtener un valor de un tipo dado.

Evaluar: Calcular un valor realizando un conjunto de operaciones sobre unos valores dados.

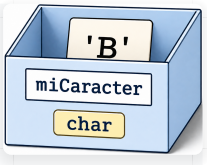
Al estar acostumbrados a leer de izquierda a derecha, que el operador de asignación mueva un valor de derecha a izquierda puede parecer extraño al principio. Sólo debemos recordar leer el = como “se establece igual a” u “obtiene”; de ese modo parece más natural.

El valor que se asigna a la variable debe ser del mismo tipo que la variable. Dadas las declaraciones:

```
String nombre;   // El nombre de una persona
String apellido; // El apellido de una persona
String titulo;   // El título de una persona, como Dr.
char inicial;    // Inicial del nombre de una persona
char miCaracter; // Un lugar para almacenar una letra
```

las siguientes sentencias de asignación son válidas:

```
nombre = "Antonio";  // Literal de cadena asignado a String
apellido = "Cepero"; // Literal de cadena asignado a String
titulo = "Profesor"; // Literal de cadena asignado a String
inicial = 'A';       // Literal de caracter asignado a char
miCaracter = 'B';    // Literal de caracter asignado a char
```



Declaraciones de asignación no válidas con explicación:

```
inicial = "A.";      // "A." es una cadena e inicial es char
nombre = Antonio;    // Antonio es un identificador no declarado
"Edison" = apellido; // Literal a la izquierda de =
apellido = ;         // Falta expresión a la derecha de =
```

Expresiones de cadena

No podemos realizar operaciones matemáticas con cadenas pero Java nos proporciona la clase `String` con una operación especial denominada concatenación, que utiliza el operador suma (+). La concatenación de dos cadenas proporciona una nueva cadena que contiene los caracteres de las dos. Por ejemplo, dadas las declaraciones:

```
String ciclo;
String p1 = "Desarrollo de Aplicaciones";
String p2 = "Multiplataforma";
```

Podemos escribir:

```
ciclo = p1 + " " + p2;
```

cuyo resultado es la asignación a `ciclo` de la cadena `"Desarrollo de Aplicaciones Multiplataforma"`[24]. El orden de las cadenas en la expresión determina cómo aparecen en la cadena resultado. Si, por ejemplo, hubiéramos escrito:

```
ciclo = p2 + p1;
```

el resultado hubiera sido la asignación a `ciclo` de la cadena `"MultiplataformaDesarrollo de Aplicaciones"`[25]. La concatenación puede utilizar tanto literales, como constantes con nombre y variables. Por ejemplo:

```
ciclo = "Estudiamos: " + palabra3 + " "
        + PALABRA5 + " en el Pablo Serrano";
```

tiene como resultado la asignación a `ciclo` de la cadena `"Estudiamos: Desarrollo de Aplicaciones Multiplataforma en el Pablo Serrano”"`.

En el caso anterior hemos combinado identificadores y literales. Por supuesto, resulta posible usar solo el literal; y, si como sucede en esa asignación, no cabe en una línea, podemos separarla en varias y concatenarlas, como en:

```
ciclo = "Estudiamos: Desarrollo de Aplicaciones Multiplataforma" +
        " en el Pablo Serrano";
```

24. Notad que hemos añadido un espacio en blanco, una cadena que solo contiene un espacio en blanco, siendo el resultado la concatenación de las tres, en el orden en que aparecen en la expresión.

25. En este caso no hemos añadido el espacio y no aparece en el resultado.

En muchas ocasiones nos encontraremos con que queremos añadir algunos caracteres a una cadena existente. Imaginemos que `ciclo` contiene `"Estudiamos "` y queremos añadirle el nombre del Centro. Podemos utilizar una sentencia como:

```
ciclo = ciclo + " en el Pablo Serrano";
```

La sentencia recupera el valor de `ciclo` de la memoria, añade el literal para crear una cadena nueva (`"Estudiamos en el Pablo Serrano"`) y asigna esa nueva cadena a `ciclo`. La nueva cadena reemplaza a la anterior.

La concatenación solo es válida para valores de la clase `String`. Si intentamos concatenar el valor de uno de los tipos primitivos de Java con una cadena, Java convertirá el valor en la cadena equivalente y realizará la concatenación. Por ejemplo:

```
String resultado;
resultado = "El cuadrado de 6 es " + 36;
```

asigna a `resultado` la cadena `"El cuadrado de 6 es 36"`. Java convierte el literal entero `36` en la cadena `"36"` antes de realizar la concatenación.

Expresiones de inicialización

Ahora que hemos visto qué es una expresión, podemos generalizar la lista de identificadores de la sintaxis de declaración de campo, sustituyendo el literal por una expresión, como aparece en la asignación:

```
IdentifierList:
  Identifier [= Expression] {, Identifier [= Expression]}
```

Esta definición convierte en válida la declaración:

```
String resultado = "El cuadrado de 6 es " + 36;
```

No obstante, y aunque hora pueda parecer un poco confuso, la plantilla sintáctica de una expresión es:

```
Expression:
  AssignmentExpression

AssignmentExpression:
  Assignment
```

Entrada y salida

Conocemos ya suficiente sintaxis de Java para decirle al ordenador que asigne valores a variables y que concatene cadenas, pero no nos dirá los resultados porque aún no

sabemos cómo decirle que nos los muestre. Como si le preguntamos a alguien si sabe qué hora es y nos responde sonriente que sí, que lo sabe.

Llamadas a métodos

Los métodos (operadores asociados con objetos en sentido abstracto) se implementan como subprogramas[26] que están llamados a realizar algún conjunto de acciones predefinido. Ya vimos que una llamada a un método es una forma de sentencia ejecutable de Java. Escribimos la declaración de llamada simplemente especificando el nombre del método, seguido de una **lista de parámetros** entre paréntesis. La llamada provoca que el control del ordenador pase a las instrucciones del método, siendo los valores que se le dan como argumento una forma de comunicación con el subprograma[27]. Cuando el método ha completado su tarea, el control vuelve a la sentencia que sigue a la llamada.

Lista de parámetros: Mecanismo para la comunicación de datos a un subprograma.

Debemos tener en cuenta que los argumentos son opcionales, pero los paréntesis no. A menudo escribimos declaraciones de llamada de la siguiente forma:

```
nombreMetodo();
```

Un sinónimo del término llamar es invocar. Decir que se invoca un método es otra forma de indicar que se realiza la llamada.

Método print

Java proporciona un objeto que representa un dispositivo de salida, para mostrarnos cosas; de forma predeterminada, la pantalla. Podemos enviar mensajes a este objeto pidiéndole que imprima algo en la pantalla. El nombre del objeto es `System.out`[28] y uno de los mensajes que podemos enviar (método que podemos aplicar) es `print`. Por ejemplo:

```
System.out.print("María" + " " + "Pilar");
```

muestra

```
María Pilar
```

en consola, una ventana de la pantalla. Hay varias cosas a tener en cuenta acerca de esta sentencia: invocamos el método (enviamos el mensaje al objeto) colocando el

26. Recordemos las estructuras básicas: secuencia, selección, bucle y subprograma.

27. Veremos que los datos pueden ser dados a un subprograma pero también pueden ser recibidos resultados desde el subprograma.

nombre del método a continuación del nombre del objeto, con un punto en el medio. El "algo" que se imprimirá es una expresión de cadena que sirve como parámetro del método. La cadena aparece entre paréntesis. ¿Qué imprime el siguiente fragmento de código?

```
System.out.print("María");
System.out.print(" ");
System.out.print("Pilar");
```

La respuesta correcta es que ambos fragmentos de código imprimen lo mismo. Los sucesivos mensajes enviados a través del método de impresión muestran las cadenas una al lado de la otra, en la misma línea. Si deseamos pasar a la línea siguiente después de imprimir la cadena, cosa bastante habitual, podemos utilizar el método `println`. Es similar al método `print` pero tiene la característica añadida de causar que la siguiente salida se imprima en la siguiente línea. Por ejemplo, el fragmento de código:

```
System.out.println("María");
System.out.println(" ");
System.out.print("Pilar");
```

muestra

```
María

Pilar
```

El método `println` no pasa a la siguiente línea hasta que se imprime la cadena. La segunda línea contiene un espacio en blanco, no es la cadena vacía. También podemos imprimir variables en lugar de literales. Por ejemplo:

```
String suNombre = "María Pilar";
System.out.println(suNombre);
```

imprime en la pantalla exactamente lo mismo que la sentencia:

```
System.out.print("María Pilar");
```

Hay una diferencia, sin embargo. Si se usa la última sentencia, la siguiente cadena comenzaría en la misma línea, a continuación del carácter *r*. Si se utilizan la declaración y la sentencia -la que usa el método `println`- anteriores, la cadena del siguiente mensaje a `System.out` comenzaría en la siguiente línea.

28. El objeto representa la salida estándar, que suele corresponderse con la pantalla y que está preparado para ser utilizado cuando el programa lo necesite.

Bibliotecas

Una biblioteca (library[29]) es un código desarrollado con funciones listas para usar, que realizan tareas específicas. En lugar de tener que escribir cada línea de código desde cero, podemos utilizar una biblioteca para aprovechar el trabajo previo de otros programadores y acelerar nuestro propio desarrollo.

Podemos pensar en una biblioteca como en una colección de herramientas utilizable en nuestros programas. Cada herramienta (método o clase) realiza una tarea específica. Por ejemplo, si necesitas trabajar con fechas y horas, en lugar de escribir todo el código para manejarlas, puedes utilizar una biblioteca que ya ha sido creada por otro programador. Esto ahorra tiempo y esfuerzo, además de garantizar el uso de código probado y confiable.

Las bibliotecas suelen ser desarrolladas por la comunidad para resolver problemas comunes de programación. Pueden cubrir una variedad de áreas, como manipulación de datos, interacción con bases de datos, diseño de interfaces gráficas y más. Al utilizar bibliotecas, puedes concentrarte en resolver los aspectos únicos de tu proyecto en lugar de reescribir código estándar una y otra vez.

Java, como otros lenguajes de programación modernos, especialmente los orientados a objetos, incorpora una amplia biblioteca de clases. Dentro de la biblioteca, las clases se agrupan en lo que se denominan paquetes (*packages*). Para utilizar sus componentes debemos usar declaraciones de importación: sentencias que indican al compilador de Java qué paquetes de la biblioteca utiliza nuestro programa.

> **Paquete:** Colección de clases que puede ser importada por un programa.

Cuando mostramos por primera vez la plantilla de sintaxis para una aplicación Java, mencionamos que puede incluir, opcionalmente, una declaración de importación. Las declaraciones de importación pueden formar parte del principio de cualquier declaración de clase. Aquí está el diagrama de sintaxis para tal declaración:

```
ImportDeclaration:
  SingleTypeImportDeclaration

SingleTypeImportDeclaration:
  import PackageOrTypeName ;

PackageOrTypeName:
  Identifier
  PackageOrTypeName . Identifier
  PackageOrTypeName . *
```

En la práctica, una declaración de importación comienza con la palabra **`import`**, seguida del nombre de un paquete o clase. Un paquete puede ir seguido de subpaquetes

separados por un punto (.), terminado con el nombre de una clase o el carácter asterisco (*). La declaración finaliza con un punto y coma (;). Si solo necesitamos utilizar una clase de un paquete en particular, podemos hacerlo indicando su nombre (*`Identifier`*) en la declaración; pero, si queremos usar múltiples clases del paquete, el asterisco le dice al compilador que importe todas las clases del paquete.

¿Para qué podríamos querer utilizar la primera forma si el asterisco es más sencillo de escribir y tiene el mismo efecto? En el primer caso se importa solo la clase que se necesita, mientras que con el asterisco podemos estar importando clases que no deseamos importar y que, como entenderemos con la práctica, pueden colisionar con nuestras propias clases.

Entrada

Los programas necesitan datos sobre los que operar. En nuestros ejemplos, hemos escrito todos los valores de los datos en el propio código, utilizando constantes literales y con nombre. Si ésta fuera la única forma en la que podrían facilitarse datos a los programas, estaría severamente restringido lo que un programa podría hacer.

Una de las mayores ventajas de usar un ordenador es poder escribir un programa que puede utilizarse con muchos conjuntos diferentes de datos. Esto significa, por supuesto, que los datos no pueden escribirse en el código. El programa y los datos deben estar separados hasta el momento en que se ejecuta el programa. En ese momento, las llamadas a métodos en el código harán que el ordenador coloque valores de los datos en las variables del programa. Después de almacenar estos valores en las variables, el código puede seguir y realizar los cálculos correspondientes.

El proceso de colocar valores desde un conjunto de datos externo en las variables del programa se llama entrada. El conjunto de datos para el programa puede venir desde un dispositivo de entrada o desde un archivo en un dispositivo de almacenamiento auxiliar. Por ahora, todas las entradas se harán desde un terminal a través de la entrada estándar, representada por el objeto `System.in`.

Desafortunadamente, Java no hace que introducir datos desde `System.in` sea tan simple como lo es mostrarlos a través de `System.out`. `System.in` es un objeto muy primitivo, diseñado para servir de base a otros objetos más sofisticados. Con `System.in`, podemos ingresar un solo byte o una serie de bytes pero, para que nos sea útil, estos datos deben convertirse en una cadena u otro tipo de dato.

Buscamos a través de la documentación de la biblioteca de Java y encontramos una clase llamada `Scanner`, útil para separar los elementos de la línea de entrada en elementos individuales, en función de su tipo de datos. Para separar los datos, por defecto, utiliza lo que se conoce como blancos, que son los caracteres espacio en blanco y tabulador.

29. Es común que, en lugar de este término, que es el correcto, lo encontréis como librería: falso amigo del inglés.

Para poder utilizar esta clase `Scanner`, en primer lugar, debemos importarla para que nuestro código sepa cómo hacerlo. Para ello, utilizaremos la sentencia de importación correspondiente:

```
import java.util.Scanner;
```

Recordemos que dicha sentencia deberá aparecer antes de la declaración de clase. Necesitamos ahora un objeto de la clase `Scanner`, es decir, necesitamos instanciar un objeto de la clase `Scanner`. Hemos indicado que usamos el operador **new** para instanciar objetos. Para ello, escribimos la palabra reservada **new**, seguida del nombre de la clase y una lista de argumentos. Por ejemplo:

```
Scanner teclado; // Un escáner de texto asociado al teclado
teclado = new Scanner(System.in);
```

El código que sigue a **new** se parece mucho a una llamada a un método. De hecho, es una llamada a un tipo especial de método llamado constructor. Cada clase tiene, al menos, un método constructor. El propósito de un constructor es preparar un nuevo objeto para su uso. El operador **new** crea un objeto de la clase pero lo crea vacío, sin datos, y luego invoca al método constructor, que puede completar los campos del objeto o realizar cualquier acción necesaria para que el objeto sea utilizable. Por ejemplo, el código anterior, crea primero un objeto de la clase `Scanner` y hace que utilice `System.in` como origen de datos. El nuevo objeto se asigna a la variable `teclado`. Los constructores se invocan solo a través del operador **new**; no podemos escribirlos como llamadas a métodos normales.

Los constructores tienen una característica especial que requiere una explicación más detallada. La llamada al método constructor no está precedida por un nombre de objeto o de clase como en el caso de System.out.`print`, por ejemplo. No tenemos que preceder el nombre del constructor con el de la clase porque su propio nombre le dice a Java a qué clase pertenece: como el constructor crea un objeto antes de que se asigne a un campo o variable, no podemos asociarlo con un objeto en particular.

El nombre del constructor no sigue nuestra regla habitual de que los nombres de los métodos empiecen por minúscula. Por convención, todos los nombres de clases (en la biblioteca de Java) comienzan con una letra mayúscula y el nombre del constructor debe escribirse exactamente igual que el nombre de la clase.

Método next

La clase `Scanner` nos proporciona otro útil método, llamado `next`. Su nombre es descriptivo de su función: devuelve lo «siguiente» que haya en la entrada (como una cadena). Aquí hay un ejemplo de uso:

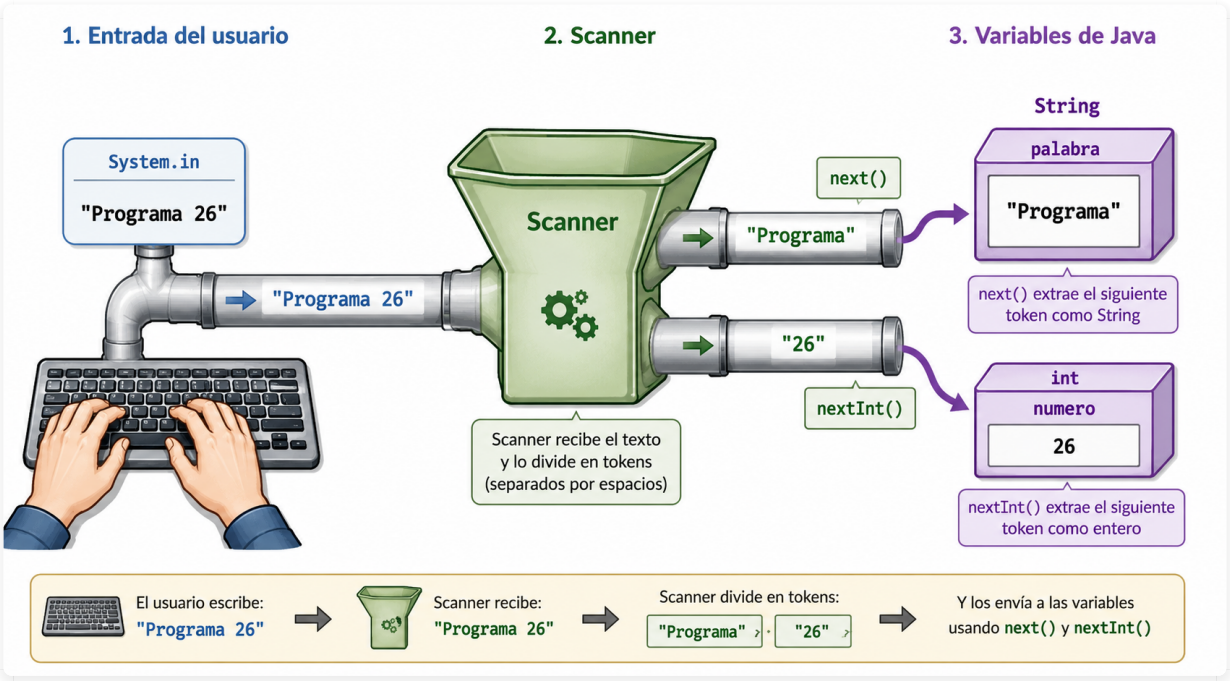
```
String unaLinea; // Un String
unaLinea = entrada.next();
```

La llamada a `next` es bastante diferente de la llamada a `print`. En la sentencia de lectura utilizamos una asignación, mientras en la de escritura no. Java admite dos tipos de métodos: métodos con retorno de valor (**funciones**) y métodos nulos (**procedimientos**). Una función se invoca dentro de una expresión y, cuando retorna, devuelve el valor que ha calculado tomando su lugar en la expresión y pudiendo después ser utilizado para la asignación u otro cálculo posterior. Un procedimiento no devuelve ningún valor: lo invocamos como una sentencia separada y, cuando ha terminado su ejecución, continúa con la sentencia siguiente.

Función: Método invocado desde una expresión y que devuelve un valor que puede ser utilizado en la misma.

Procedimiento: Método invocado en una sentencia y a cuyo regreso la ejecución continúa con la sentencia siguiente.

Si la línea tiene más de un elemento, como varias palabras, `next` nos devolverá cada palabra, una tras otra. Si lo que queremos una cadena con todas ellas, podemos utilizar el método `nextLine`, que nos devuelve lo que haya en la entrada hasta el final de la línea. Además, nos proporciona otros métodos, útiles para la lectura de datos de otros tipos, no solo de cadenas, como `nextInt`, por ejemplo.



Entrada y salida interactivas

Un programa interactivo es aquél en el que el usuario se comunica directamente con el ordenador. Muchos de los programas que escribiremos serán interactivos. Hay algunas reglas para escribir programas interactivos. Esto implica, principalmente, suministrar instrucciones claras al usuario sobre los datos a introducir.

Antes de leer los valores en un programa interactivo, el programa debe imprimir un mensaje para explicar al usuario qué ha de introducir. Esos mensajes se denominan *indicativo de petición de entrada*. Sin ellos, el usuario no sabrá qué debe proporcionar al programa. Un programa que no solicite valores de entrada será muy difícil de utilizar, si es que resulta posible el hacerlo. Según las circunstancias, el programa también deberá imprimir los valores que se han escrito para que el usuario pueda verificar que lo introducido es correcto. La impresión de los valores de entrada se denomina *impresión del eco*.

El siguiente segmento de código muestra el uso adecuado de los indicativos de petición cuando solicita un nombre:

```
System.out.print("Introducir el nombre: ");
nombre = teclado.nextLine();   // Obtener nombre
System.out.print("Introducir apellido: ");
apellido = teclado.nextLine(); // Obtener apellido
```

El siguiente segmento de código muestra la impresión del eco:

```
System.out.println("Usted introdujo el nombre: "
                   + nombre + " " + apellido);
```

Aunque la impresión del eco puede parecer redundante en una pantalla, es crucial para la verificación de los datos de entrada.